zenodo:10.5281/zenodo.23918522 [cs.RO] Oct 2026 DOI: 10.5281/zenodo.23918522

Getting SmolVLA to Run on a Low-Cost Arm: Training, Remote Inference, and a First Language-Conditioning Probe

Iliya Radulov[1]

[1]Independent researcher

October 2026

Abstract

We report how a small vision-language-action model (SmolVLA) was brought, step by step, onto a self-built SO-101 arm that had previously been driven by an ACT policy. The paper covers three things. First, the fine-tuning recipe and the two training runs that produced the main checkpoint used in all 8-position rollouts. Second, the inference problem: the model ran at 0.55 policy requests per second on the arm's Raspberry Pi 5, about 54 times too slow, and moving it to a desktop GPU served over the network through LeRobot's asynchronous inference brought the policy-request rate to about 6 Hz while leaving the Pi nearly idle and the arm executing a 20-action chunk between requests. Third, a small probe of language conditioning, in which four tiny datasets that differ in their instruction (and in the demonstrated trajectory for each instruction) showed that the policy's behavior changes with the instruction in the intended direction, including for a paraphrase whose exact wording was not in training. The one controlled quantitative comparison we run on the 8-position grid is between the two execution modes on the same machine, same framework version, and same arm: synchronous inference scored 40/80 and RTC scored 41/80, a difference of 1.25 percentage points (Fisher's exact $p = 1.00$). More broadly, we exercised the same recipe and the same checkpoint across three different LeRobot clones, three client machines, and two physical follower arms during the engineering work in Sections 4 and 5, and the pipeline continued to produce functional policies; we flag this as an engineering-reproducibility observation rather than a statistical claim, because the full 8-position protocol was only run under one of those combinations. We do not report a controlled comparison with ACT; the comparison in the final section mixes model and conditions, and the per-position diagnosis is left to a follow-up paper. This is a single-operator study with small samples; it is meant as a reproducible record for others on similar budgets.

1 Introduction

This is the fourth paper from one project. The first studied trigger-based attacks on robot policies [1]. The second traced a policy that trained without errors and did nothing useful to a distribution mismatch in the camera footage, and quantified the effects of lighting and background on a pick-and-place policy trained with ACT [4], which reached 86.3% success on trained positions [2]. The third is an informal companion about hardware, software and the mistakes behind both [3].

ACT was always meant as a stepping stone. The goal is a policy that takes a natural-language instruction, because the robots we want to build next will need more than one task. SmolVLA [5] is a compact open vision-language-action model that is designed to be trained and run on consumer

zenodo:10.5281/zenodo.23918522 [cs.RO] Oct 2026 DOI: 10.5281/zenodo.23918522

hardware, and it is supported by the same LeRobot framework we already used. We expected the switch to be quick. It was not, and the reasons are mostly not about the model.

We make a deliberately modest claim. Starting from nothing, we obtained a SmolVLA policy that completes the task in roughly half of its attempts on the positions it was trained on, with a language link that responds to the instruction in a small probe. The two full 8-position baselines we did run — synchronous and RTC execution, same machine, same framework version, same arm — came out at 40/80 and 41/80 and are not statistically distinguishable. The remaining machine, version, and arm changes in this paper are exercised in the engineering sections (pipeline reruns, cadence measurements, language probe) rather than in a full 10-trial baseline, so we treat consistency across them as engineering reproducibility, not a statistical reproducibility claim. We do *not* claim that SmolVLA is better than ACT; in our setup it is currently worse on the same task, and the comparison is not like for like. The unfinished part, namely why the policy fails in the other half and what can be done about it, is the subject of a follow-up.

Contributions.

- A complete, reproducible account of the training recipe, with its commands and the mistakes we avoided or made along the way (Section 3).
- Measured policy-request rates for three ways of running SmolVLA on a Raspberry Pi 5-based arm, and the architectural conclusion that follows (Section 4).
- A small, honest probe of language conditioning with its design, its results, and its limits (Section 5).
- A first look at real-time chunking and two full 8-position baselines (synchronous and RTC) that cannot be distinguished at the aggregate level, plus an engineering-reproducibility observation across the machines, framework versions and arms exercised during the engineering work (Sections 6 and 7).

2 Setup

Hardware and names used throughout. The arm is an SO-101 with a leader arm for teleoperation. Two USB cameras are used: a main camera in front of the arm (the placement found in the findings paper [2]) and a wrist camera. In the 96-episode main dataset the main camera recorded at 640×480 and the wrist camera at 320×240, both at 30 fps; the four small language-conditioning probe datasets (Section 5) were both recorded at 640×480 at 30 fps. Three machines appear throughout the paper and are referred to by the names in this paragraph: **Pi 5 (`lab`)**, a Raspberry Pi 5 at the arm rig used for teleoperated recording and for the asynchronous robot client; **Mac (M4)**, an Apple-silicon laptop used for probe recordings and for the rollouts in Sections 5 and 7 (device `mps`); and the **ML Desktop**, an x86 workstation with an RTX 5060 Ti (16 GB) used for training and for the asynchronous-inference policy server. Two physical follower arms appear: the **original follower arm** (the one used for the ACT policy in [2] and for most of the engineering work here) and the **Kiwi follower arm**, a spare of the same design that replaced the original after a motor failure on 28 September and that was used for the two 8-position baselines in Section 7. The machines are described in more detail in the companion field notes [3].

A follower safety cap. All results below are collected with the follower arm's `Maximum_Acceleration` and `Acceleration` registers capped at 99 (down from the LeRobot default of 254), applied as a one-line patch to `so_follower.py` at the `configure()` call (the patch file lives in the companion repository, see Section 9). The cap was adopted after a hardware incident: during a session with the default setting the servo power bus collapsed from a steady 12.4 V to about 3.3 V during a hard-acceleration move, which dropped communication with all six motors simultaneously and consistently with a power-rail brownout rather than a per-motor fault. 99 was the largest value we tested at which such collapses did not recur over the subsequent sessions; we did not search further for the smallest safe value. The cap must be applied by patching the driver because `configure_motors()` is called from `so_follower.py`'s `configure()` on every `connect()` and resets the registers to the hard-coded default, so setting the registers from outside the driver has no persistent effect. All success-rate numbers in Sections 6 and 7 are collected under this cap; the full diagnosis of the brownout and of the per-position failures is left to the follow-up paper.

Data. All main experiments use one dataset: 96 teleoperated episodes (56,856 frames) of picking a yellow cubic sample and placing it in a target. The episodes cover 8 positions × 4 orientations × 3 repetitions, so every position has the same number (12) of episodes. They were recorded in three batches of 32, 31 and 33 episodes; the last episode of the second batch was lost to a crash while the video encoder was closing, and the third batch was lengthened by one to compensate. The batches were merged into one dataset with the single task string "Pick up the object and place it in the target." This is the same dataset the ACT policy was trained on [2].

Evaluation. For SmolVLA, a rollout runs for at most 40 s from a fixed start pose; for ACT (the reference comparison in Section 7) the rollout duration used in [2] is 20 s. We test the 8 trained positions with 10 trials each. A trial counts as a success if, within the time budget, the arm has placed the object inside the target region as scored by the operator (same rule as the findings paper). A trial that completes only after a second attempt within the time budget counts as success equally with a first-attempt completion; no partial credit is given, and a timeout without placement counts as failure. Rollouts of the two policies were not run under identical conditions (different computer, different arm build, different control software, and twice the rollout time for SmolVLA), which matters for the comparison in Section 7.

Experiments at a glance. The engineering work in Sections 4–7 spans three client machines, two physical follower arms and three checkpoints. Table 1 is a single reference map of what ran where; the diagram in Section 7 (Figure 1) shows the machine/arm wiring for the main baselines specifically.

zenodo:10.5281/zenodo.23918522 [cs.RO] Oct 2026 DOI: 10.5281/zenodo.23918522

Table 1: Overview of SmolVLA (and reference ACT) experiments in this paper. "Main 20k" refers to the SmolVLA checkpoint of Section 3 (fine-tuned from `smolvla_base` on the 96-episode dataset); "Probe 3k"/"Probe 20k" refer to the two probe checkpoints of Section 5 (fine-tuned on the 40-episode probe dataset). The ACT row is the reference comparison from [2].

Experiment	Checkpoint	Dataset	Arm	Client	Execution	Trials
Pi CPU cadence (§4)	Main 20k	96-ep	original	Pi 5 (`lab`) local CPU	sync	1 run
Patched sync (§4)	Main 20k	96-ep	original	Pi 5 → ML Desktop GPU	sync + custom patch	1 run, 2,513 ticks
Async cadence (§4)	Main 20k	96-ep	original	Pi 5 → ML Desktop GPU	async, chunk 20	informal
Chunk sweep (§4)	Main 20k	96-ep	original	Pi 5 → ML Desktop GPU	async, chunks 10/15/20/30	4 chunk sizes, qualitative
Lang. probe, 3k (§5)	Probe 3k	40-ep probe	original	Mac (M4, mps)	sync	3 × 4 instr.
Lang. probe, 20k (§5)	Probe 20k	40-ep probe	original	Mac (M4, mps)	sync	3 × 4 instr.
Sync baseline (§7)	Main 20k	96-ep	Kiwi	Mac (M4, mps)	sync	8 pos × 10 = 80
RTC baseline (§7)	Main 20k	96-ep	Kiwi	Mac (M4, mps)	RTC	8 pos × 10 = 80
ACT reference (§7)	ACT [2]	96-ep	original	Pi 5 (`lab`) local	—	8 pos × 10 = 80

"Original" and "Kiwi" are the two follower arms introduced in Section 2.

3 Training

3.1 The recipe

We fine-tune from the public base checkpoint `lerobot/smolvla_base`. The command is short; the details are in the flags.

```
lerobot-train \
  --policy.path=lerobot/smolvla_base \
  --dataset.repo_id=local/so101_pi5_act_v3_merged \
  --dataset.root=<path to the merged dataset> \
  --rename_map='{"observation.images.main":    "observation.images.camera1",
                 "observation.images.gripper": "observation.images.camera2"}' \
  --policy.empty_cameras=1 \
  --batch_size=16 --steps=20000 \
  --policy.device=cuda --policy.push_to_hub=false --wandb.enable=false \
  --output_dir=outputs/train/smolvla_full_run
```

Two points are easy to miss. The base model expects its cameras under fixed names, so our own camera names have to be mapped with `--rename_map`; the same map must be given again at rollout, because it is not stored in the checkpoint. And the base model has a third camera slot that we do not fill. We first worked around this with a stand-in webcam, and later found that all visual features of the model use identity normalization, so the empty slot needs no statistics and no stand-in.

zenodo:10.5281/zenodo.23918522 [cs.RO] Oct 2026 DOI: 10.5281/zenodo.23918522

3.2 Runs

Table 2 lists the two runs on the 96-episode dataset. The 5,000-step run was a pipeline check. All rollouts reported below use the 20,000-step checkpoint of the second run.

Table 2: SmolVLA fine-tuning runs on the 96-episode dataset (RTX 5060 Ti).

Run	Steps	Batch	Time	Epochs	Final loss
Pipeline check	5,000	8	≈24 min	0.70	0.13–0.14
Full run	20,000	16	3 h 03 min	5.63	≈0.072

The full run processed 320,000 samples, which is 5.63 passes over 56,856 frames, at about 1.8 steps per second and 5.9 GB of GPU memory. The loss flattened around 0.07 from roughly step 17,000, while the learning rate decayed to 2.5×10^{-6}. For comparison, our ACT policy was trained for 50,000 steps on the same data. The SmolVLA budget is therefore small, and we did not test whether longer training changes the picture. We refer to this checkpoint as the *main* 20,000-step checkpoint in the rest of the paper. All results in Sections 4, 6 and 7 come from this one checkpoint; the language-conditioning probe in Section 5 uses two *separate* checkpoints (3,000 and 20,000 steps) trained from the same base model but on the 40-episode probe dataset only, and should not be confused with the main checkpoint used for the 8-position baselines.

3.3 Two rules we adopted

Merge the data, then train once. When a second task is added, the natural move is to continue training the existing checkpoint on the new data. A bachelor's thesis that used the same robot, model and framework fine-tuned SmolVLA sequentially on three 50-episode datasets and saw performance rise after the second step (10% to 60% and 80% on two similar tasks) and then collapse to 0% on the final task, with earlier tasks degraded as well [10]. The author reads this as catastrophic forgetting and notes that the evidence is ambiguous and not reproducible from day to day; a model fine-tuned from the base checkpoint on a single 100-episode dataset did better than the 150-episode sequential one. We treat this as a warning and not a proof, and adopted a precaution: merge all task datasets into one and fine-tune once from the base model. In the installed LeRobot version, passing a list of datasets to `--dataset.repo_id` does not work: the multi-dataset code path is disabled and the argument is a plain string. The datasets must be merged physically first, which we did with LeRobot's own aggregation function and then verified, since a wrong task index would silently invalidate a language experiment.

Keep diagnostic data out of the training folder. Probe recordings (Section 5) live in a separate directory tree from production recordings, so that they cannot be merged into a real training run by accident.

4 Where the Model Runs

We use four distinct terms for "rate" in this section, which have been confused in our own earlier notes:

- **Action execution rate**: how often the arm sends a target pose to the motors (30 Hz, fixed by the control loop).

zenodo:10.5281/zenodo.23918522 [cs.RO] Oct 2026 DOI: 10.5281/zenodo.23918522

- **Policy-request rate**: how often the client sends an observation to the policy and receives an action back. Equal to the action execution rate under synchronous execution with no chunking.
- **Replanning rate / chunk rate**: in asynchronous execution, how often a new chunk of actions is returned by the server.
- **Synchronous tick latency**: for the synchronous engine only, the mean time between the start of a control tick and the action it produced, equal to the inverse of the policy-request rate under synchronous execution.

The headline number for a synchronous local run ("the Pi cannot do it" below) is the policy-request rate; for the asynchronous system it is also the policy-request rate, which differs from the action execution rate because the arm executes a chunk between requests.

4.1 The Pi cannot do it

The first rollout ran SmolVLA directly on the Pi 5's CPU under synchronous execution. The policy-request rate was 0.55 Hz against a target of 30 Hz, about 54 times too slow. The cause is compute, not memory: all Pi 5 memory variants share the same processor, so a variant with more RAM would not help. Strangely, the task still completed in a rollout, which is one reason the problem went unnoticed for a long time (see the field notes [3]).

4.2 Offloading

We moved inference to the ML Desktop in three steps.

Attempt 1: forward the hardware. Instead of splitting the code, we tried to attach the Pi's USB devices to the ML Desktop over the network (USB/IP), so that the unmodified rollout command could run there. Before building anything we checked the network (173 Mbit/s over Wi-Fi, no retransmissions) and that both cameras stream compressed MJPEG, which left ample headroom. All three devices attached. The cameras worked. The arm did not: the very first write to the motor bus failed with "no status packet," consistently and before any motion. The arm's USB-serial adapter uses a write-then-reply handshake whose timing the extra network hop apparently does not preserve, whereas video tolerates it. We used ACT for this test on purpose, so that a failure could only mean the connection and not the policy. The attempt also taught us the rollout code well enough to make the next one possible.

Attempt 2: one patched line. The synchronous rollout engine calls the policy once per control tick on an observation that is already preprocessed. We wrapped that one call so that, if an environment variable is set, the observation is sent over HTTP to a small server on the ML Desktop that holds the model and returns the action. Everything else stayed untouched. This worked: the policy completed a full grasp-and-place through the remote pipeline for the first time, at a policy-request rate of 3.02 Hz over 2,513 ticks (Table 3; the synchronous tick latency was 0.33 s mean, 1.16 s worst case). The loop spent 98.9% of its time in the remote call. Our diagnosis is that every tick shipped uncompressed floating-point images of roughly 4–5 MB, which at the measured bandwidth accounts for about 200 ms of the 327 ms mean. We did not implement image compression.

zenodo:10.5281/zenodo.23918522 [cs.RO] Oct 2026 DOI: 10.5281/zenodo.23918522

Attempt 3: the official asynchronous system. LeRobot ships a policy server and a robot client. The server runs on the ML Desktop and loads the policy when a client connects; the client runs on the Pi 5 (`lab`) and sends observations and executes actions. It needed one extra install (`lerobot[async]`) on both machines. The key property is chunking: the server returns a chunk of actions per call and the client executes the chunk locally at full rate, so the network and compute cost is paid once per chunk and not once per tick. With 20 actions per chunk and a request threshold of 0.5, we measured a policy-request rate of about 6.1 Hz, with the Pi at 5–6% CPU; the action execution rate was unchanged at the 30 Hz control-loop target during each chunk's execution. The arm moved smoothly and, in informal tests, completed the task repeatedly, including once from a position it was not trained on. Because it needs no custom code, we adopted it and kept the patch only as a documented fallback.

Table 3: Inference placement for SmolVLA (Pi 5 client, desktop GPU). "Rate" is the policy-request rate (observations sent to the policy per second). The Pi CPU and temperature values refer to the Pi during the rollout.

Setup	Policy-request rate	Mean tick latency	Pi CPU	Pi temp.
On the Pi's CPU	0.55 Hz	1.83 s (worst 97.4 s)	≈100%	60–68°C
Patched sync, remote GPU	3.02 Hz	0.33 s (worst 1.16 s)	5–6%	≈50°C
Official async, remote GPU	≈6.1 Hz	—	5–6%	—

The policy-request rate of the asynchronous system is how often the loop returns to the GPU for a new plan, not how often the arm moves; between requests the arm executes the 20-action chunk continuously at the 30 Hz action execution rate. This is why it felt much faster than the number suggests. A caution on reading its log: the printed counter is the position in the action queue, not a frame count, and the reported average rate is the trustworthy figure.

4.3 Tuning, and what it did not fix

Two parameters matter most: the number of actions per chunk, and the threshold at which the client asks for a new chunk. A quick sweep over the chunk length (Table 4) showed a clear trade-off between rate and precision, and we chose 20. A second, later sweep over the threshold and the blending function for overlapping chunks found a slightly better-behaved configuration at 25 actions and a threshold of 0.3, at about 3.8 Hz. Higher rate and better motion pulled in opposite directions.

Importantly, the shape of the arm's approach did not change across any of the settings we tried: a sideways, snake-like approach, a gripper that opens too little, and a tendency to push the object instead of grasping. These behaviors persisted across the tested chunk-length and threshold settings, suggesting that those settings alone do not explain them. Both sweeps were informal and neither was repeated at scale, so we do not report them as tuned results.

4.4 Why we did not buy onboard hardware

For the later mobile robots the obvious question is whether a stronger onboard computer is needed. Our own measurement answers it: with inference offloaded, the Pi uses 5–6% of its CPU, and the

zenodo:10.5281/zenodo.23918522 [cs.RO] Oct 2026 DOI: 10.5281/zenodo.23918522

Table 4: Informal chunk-length sweep (4 chunk sizes) with the official asynchronous system (threshold 0.5). Observations are qualitative; this was not a success-rate study.

Actions per chunk	Policy-request rate	Behavior
10	≈12 Hz	fast but imprecise; wanders without converging
15	≈7.5 Hz	reasonably stable
20	≈6.1 Hz	best: precise, completes the task reliably
30	≈4–4.5 Hz	slower and worse; orientation problems return

extra motors of a mobile base are cheap input and output. An external study built on a similar low-cost mobile platform reports SmolVLA's maximum replanning rate on an onboard Jetson Orin Nano Super at only about 1.4 Hz, slower than our remote setup, while ACT, which has no iterative denoising step, reportedly runs at about 28 Hz on the same device [9] (we did not reproduce these numbers; they are taken from that paper). The bottleneck they identify is the iterative denoising of the flow-matching head, which does not shrink much on a small onboard GPU. For our present setup, remote inference provided sufficient performance without an onboard accelerator; we therefore deferred that purchase. We conclude that a thin client with a remote GPU is not a compromise here.

5 A First Language-Conditioning Probe

5.1 Why a probe

A language-conditioned policy is only useful if the instruction matters. A risk named *information collapse* is that when the scene alone predicts the task, as in our data with one object and one destination, a well-optimized model can learn to ignore the language entirely [8]. The authors of that paper show the effect on a benchmark: a vision-only model came within a few points of the full model on three LIBERO suites, and collapsed from 97.4% to 9.8% on the suite in which several tasks share the same scene. Our main dataset has exactly one instruction, so it cannot tell us whether language works. We therefore built a separate test.

5.2 Design

We recorded four small datasets of 10 teleoperated episodes each, with the same cube at the same position (X20/Y10 on the mat), the same cameras, and different instructions (Table 5). Each instruction was recorded together with the corresponding teleoperated trajectory, so the datasets differ both in the task string and in the demonstrated action: "push the cube to the left" is paired with leftward pushes, "push the cube to the right" with rightward pushes, and so on. What we hold constant between the four datasets is the scene (same cube, same mat position, same cameras, same lighting) and the structure of the recording session, not the action. Episodes were 15 s long at 30 fps. The four datasets were merged into one (40 episodes, 17,996 frames, four distinct task strings, verified in the task index) and used to fine-tune the base model.

The design follows one idea: each comparison changes the language and the corresponding demonstrated trajectory in a controlled way. The coarse test (grasp versus push) shows whether

zenodo:10.5281/zenodo.23918522 [cs.RO] Oct 2026 DOI: 10.5281/zenodo.23918522

Table 5: The four probe datasets (10 episodes each) and the comparison each supports.

Instruction	Role	Compared with
"pick up the yellow cube and place it in the target"	grasp	push-left (coarse)
"push the cube to the left"	push, direction A	push-right (fine)
"push the cube to the right"	push, direction B	push-left (fine)
"move the cube to the left side"	paraphrase of push-left	push-left

the model switches action type with the instruction. The fine test (left versus right) is harder because both are pushes with the same scene, so the direction can only come from the instruction. The paraphrase test catches a model that learned one exact phrase as a trigger.

We trained two checkpoints: a 3,000-step run (batch size 4, about 0.67 epochs, deliberately undertrained, converging to a loss of 0.124) and a 20,000-step run (about 4.45 epochs, converging to a loss of about 0.065), both from the same `lerobot/smolvla_base` starting point. Both were evaluated on the Mac (M4) with synchronous inference, using the rollout tool's `base` strategy on device `mps`.

5.3 Results

All results are qualitative; we ran the four instructions on the same scene a few times each and did not count successes.

- **Coarse.** The two action types were clearly distinguishable: with the pick-and-place instruction the arm reached for and tried to grasp the cube, with the push instruction it pushed.
- **Fine.** Push-left and push-right produced visibly different directions. This is the more convincing result. Execution was weaker for push-right, which sometimes ended with the arm returning toward a home-like pose; we have not found out whether that comes from the data, the kinematics, or something else.
- **Paraphrase.** In three attempts with "move the cube to the **left** side" (the paraphrase row of Table 5), the arm went left every time, even though the exact phrasing was new. The keyword "**left**" had appeared in training in the different sentence "push the cube to the **left**" (row 2 of Table 5); only the surrounding wording is new. We cannot distinguish, from this test alone, between keyword matching on "left" and a more robust response to the direction concept.
- **More training.** At 20,000 steps the direction behavior was unchanged. Grasp precision did not improve despite the lower loss: the arm went to the right place but still made contact unreliably.

5.4 What this does and does not show

The probe shows that the policy's behavior depends on the instruction, in the intended direction, and that this survives a paraphrase. It does *not* show that the policy understands language in a broad sense. It rests on one position, one object, three attempts for the paraphrase, no counted success rates, and checkpoints trained only on the 40 probe episodes. The checkpoint used in the 8-position rollouts was trained on a single instruction, so the language link has not been tested on

zenodo:10.5281/zenodo.23918522 [cs.RO] Oct 2026 DOI: 10.5281/zenodo.23918522

it at all. Finally, the plateau in grasp precision with more steps on the same 40 episodes suggests a limit set by data quantity or diversity and not by training length; we did not test this.

6 Real-Time Chunking, a First Look

Real-time chunking (RTC) [6] addresses a weakness of chunked execution: when a new chunk is predicted while the previous one is still running, the two can disagree, and the arm jerks or waits. RTC conditions the new chunk on the actions already committed to, so the transition is continuous. It ships with LeRobot and SmolVLA supports it without retraining. A related study describes a stale-observation mechanism (actions computed from observations at least one chunk old) under synchronous execution with inference delay [7]; the freeze that we observe at chunk boundaries is consistent with this picture, though the study does not describe a camera freeze itself.

We swept two parameters in a first, informal session. The execution horizon, varied from 2 to 50, had no observable effect at any value; we do not know why, and a wiring or version issue is possible. The maximum guidance weight, varied from 10 to 200, had a clear effect: a higher weight made the arm move purposefully toward the actual object and not toward a plausible but wrong place. The upper end of the useful range was not reached. These are observations from single sessions without a non-RTC baseline on the same setup; the controlled comparison is in Section 7 and the follow-up.

7 Where Things Stand

The two 8-position baselines are the only experiments in this paper that produce a 10-trial-per-position aggregate rate. Both ran on the main 20,000-step checkpoint, both on the Mac (M4), both on the Kiwi follower arm, both under LeRobot 0.6.2 dev. The only variable that differs between them is the execution mode (sync vs. RTC). The full list of every experiment and its conditions is Table 1 in Section 2; this section discusses the two baseline rows.

Figure 1 shows how the machines and arms map onto the baseline measurements and onto the engineering runs that preceded them. Synchronous inference scored 40/80 and RTC scored 41/80 (Table 6). Their Wilson 95% intervals are about 39–61% and 40–62%, and a Fisher's exact test on the 2×2 table of successes and failures gives $p = 1.00$ (two-sided): the aggregate totals are indistinguishable, differing by 1.25 percentage points. The ACT result from the findings paper is shown for reference.

Table 6: Success on the 8 trained positions (10 trials each). The ACT row was measured under different conditions and is not a controlled comparison.

Policy / execution	Successes	Rate	Wilson 95% CI
ACT (findings paper)	69/80	86.3%	77–92%
SmolVLA, synchronous	40/80	50.0%	39–61%
SmolVLA, RTC	41/80	51.3%	40–62%

We read this carefully. The ACT figure was obtained with the Pi 5 (`lab`) and the original follower arm with a 20 s rollout budget; the SmolVLA figures were obtained on the Mac (M4) with the Kiwi follower arm (built to the same design and re-calibrated) and a 40 s budget, so the gap of about 35 points mixes the model with the conditions. The sample size per position is 10, so only very large differences between positions are visible. Although the aggregate rates under

zenodo:10.5281/zenodo.23918522 [cs.RO] Oct 2026 DOI: 10.5281/zenodo.23918522

zenodo:10.5281/zenodo.23918522 [cs.RO] Oct 2026 DOI: 10.5281/zenodo.23918522

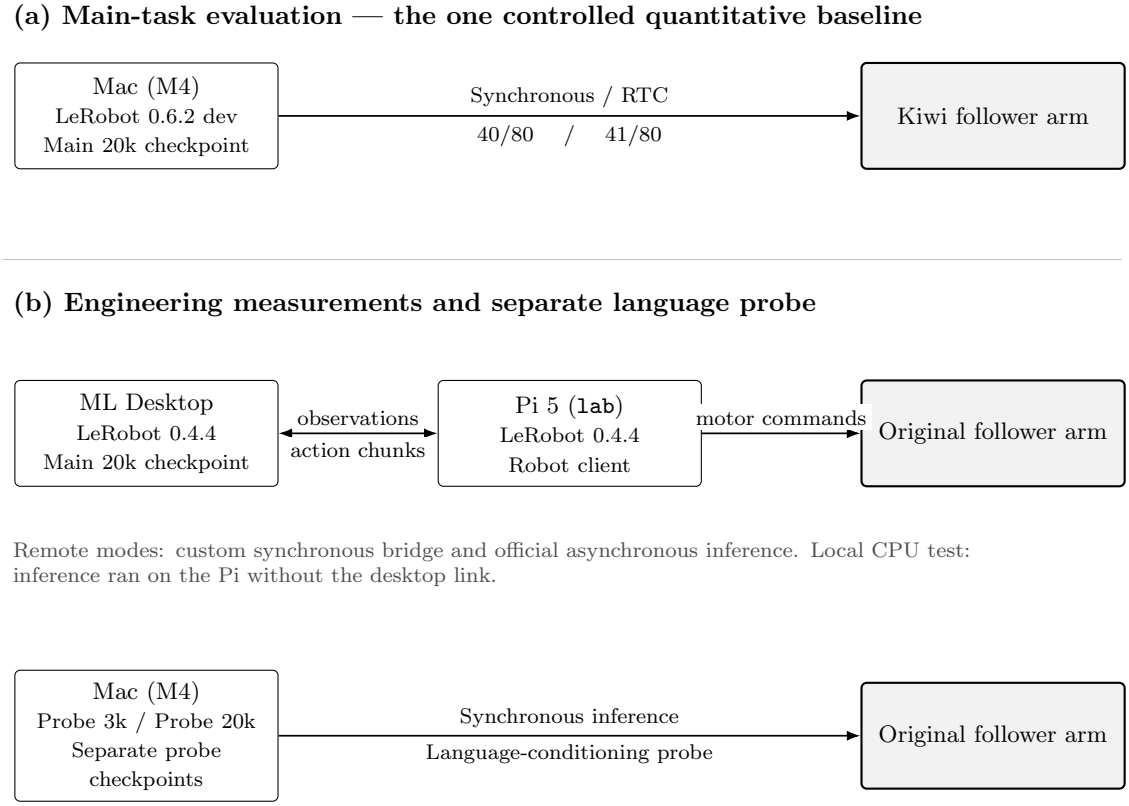


Figure 1: Experimental configurations. (a) The two quantitative SmolVLA baselines used the main 20,000-step checkpoint on the Mac (M4), controlling the Kiwi follower arm under the same LeRobot checkout; execution mode differed between synchronous inference and RTC (Section 7). (b) Deployment experiments used the Pi and original follower arm, with either local CPU inference or remote inference on the ML Desktop (Section 4). The separate language-conditioning probe used the Mac and the probe checkpoints (Section 5). Deployment runs included timing measurements and informal task demonstrations, but not the full 80-trial success-rate protocol. Repeated machine and arm boxes indicate different experimental configurations, not additional hardware.

the two execution modes were statistically indistinguishable ($p = 1.00$), the per-position pattern is not: three positions (1, 3, 7) behave near-identically under both execution modes (0 vs. 0, 10 vs. 10 and 8 vs. 8, each sync vs. RTC out of 10 trials), while positions 4 and 5 move most, in opposite directions — position 4 drops from 9/10 under sync to 5/10 under RTC, and position 5 rises from 1/10 to 7/10. Positions 2, 6 and 8 also move modestly (sync vs. RTC: 4 vs. 3, 2 vs. 4 and 6 vs. 4, each out of 10). The per-position pattern and the failure types are what we analyze in the follow-up; here we only note that the aggregate is stable at about half and that the two execution modes do not differ in it.

Beyond this sync-vs-RTC comparison, which is our one controlled quantitative result on the 8-position grid, the engineering pipeline itself was exercised across three different LeRobot clones (Table 7), three client machines (Mac (M4), Pi 5 (`lab`), ML Desktop), and two physical follower arms during Sections 4 and 5, and continued to produce functional policies throughout. We flag this as an engineering-reproducibility observation rather than a statistical claim: the full 10-trial-per-position baseline protocol was only run under one of those combinations (the solid arrow in Figure 1), and we have no 10-trial baseline under LeRobot 0.4.4 or on the original follower arm. What we actually show is that the recipe and the trained checkpoint survived substantial changes in the surrounding infrastructure without breaking down into failure modes we had not seen before.

Our assessment is therefore mixed in a useful way. ACT began at full success and then hit a wall. SmolVLA began from nothing and has reached about half, it responds to its instruction, it runs on a thin client, and the pipeline survived substantial engineering changes around it. It is less stable than ACT, and we do not yet know how much of the remaining gap is the model, the data, or the setup.

8 Small Traps from This Stage

A few problems cost us time and are not in the documentation we could find.

The tokenizer path. A checkpoint copied from the ML Desktop to the Mac (M4) failed at load time with a tokenizer error. The saved preprocessor configuration stores the tokenizer name as the bare word "tokenizer" and not as a path, so the loader cannot find the tokenizer files that are present in a subfolder. The fix is to rewrite that field to the absolute path of the subfolder once per transferred checkpoint. We hit this twice and did not establish whether it is a known upstream issue.

Camera names and slots. The rename map is needed again at rollout, and the empty third camera slot is harmless (Section 3).

Multiple datasets. The list syntax for `--dataset.repo_id` is documented in places but disabled in code; merge first.

Rollout strategy. For a quick manual test of a policy, the rollout tool's `base` strategy needs no dataset arguments, and the run length is set by the top-level duration, not by an episode count.

A moving target. The framework changed constantly while we worked, and we moved between the three machines named in Section 2 (Mac (M4), Pi 5 (`lab`), ML Desktop). Every time we set up an experiment on a machine with a different installation, some commands had been renamed, removed, or given different defaults. For example, the asynchronous client has no rename-map

zenodo:10.5281/zenodo.23918522 [cs.RO] Oct 2026 DOI: 10.5281/zenodo.23918522

option at all, so its cameras must be named as the checkpoint expects, and one of its required arguments looks optional in the help text. Much of the early confusion in our experiments came from this and not from the policy. We now treat the exact version of the framework as part of every result; Table 7 lists the clones used for the numbers in this paper. A second, older clone on the Mac, used only to calibrate arms for a mobile robot and never updated, sits at a different commit from August and behaves differently even though it shares a name — a cautionary reason to pin the commit, not just the tag or the name.

Table 7: LeRobot versions by machine, for the results reported in this paper. The short commit identifier is the first eight hex characters; full hashes are in the companion repository (Section 9).

Machine	Role	Commit	Version string
Mac (M4)	Sync and RTC baselines, Sept 30 – Oct 1	`bf9d9a18`	0.6.2 (dev)
Pi 5 (`lab`)	Async client, from 26 Oct onwards	`3f2c29ef`	0.4.4
ML Desktop	Policy server throughout	`3f2c29ef`	0.4.4

The two LeRobot versions in Table 7 are different installations of the same framework, not different checkpoints: the SmolVLA weights are identical across the three machines, and only the surrounding framework code differs. The versions are not sequential either: `3f2c29ef` (0.4.4) is an older commit than `bf9d9a18` (0.6.2 dev). The Pi and the Desktop were pinned to a known-working installation and not updated for the duration of the experiments reported here, while the Mac tracked newer async-inference fixes. As Section 7 discusses, the one controlled quantitative baseline (sync vs. RTC) was run under a single installation of the three; the other installations were exercised in the engineering work and produced qualitative evidence, not aggregate rates.

9 Limitations and What Comes Next

The main limitations are as follows. All reported SmolVLA baselines come from one main 20,000-step checkpoint trained for about 5.6 epochs on the 96-episode dataset; the language probe uses two separate checkpoints on the 40-episode probe dataset and should not be read as evidence about the main checkpoint's language link, which has not been directly tested. Samples are small (10 trials per position; three qualitative attempts per probe condition). The ACT comparison in Section 7 is not a controlled comparison: the computer, the rollout budget (20 s vs. 40 s), the arm (original follower vs. Kiwi follower) and the control software all differ. The chunk-length and threshold sweeps in Section 4 were informal and were not repeated at scale. We did not test other objects for SmolVLA. The reproducibility result in Section 7 covers the variables we actually changed (execution mode, client machine, LeRobot version, follower arm); it does not cover scene, lighting, object, camera placement, or task string.

A follow-up paper takes the 8-position baselines apart in detail: which positions fail and how (the per-position failure taxonomy and the dual recovery-mode hypothesis), which explanations we could refute (lighting, mechanical joint limits, power-rail brownout, session state), which remain open (coverage at the edge of the trained region, recovery after a missed grasp), and what we plan to try, namely tuning the real-time chunking parameters, logging cross-attention entropy as a confidence signal, and collecting more data at the weak positions. The per-position diagnosis, the dual recovery-mode question, and the ACT-vs-SmolVLA position-level comparison are deliberately

zenodo:10.5281/zenodo.23918522 [cs.RO] Oct 2026 DOI: 10.5281/zenodo.23918522

deferred to that follow-up to keep the present paper's scope on the engineering recipe and the headline baseline numbers.

Code and Data Availability

A companion repository tagged `smolvla-paper-v1` is available at https://github.com/iliya-radulov/so101-pickplace-project/tree/smolvla-paper-v1. It contains, in `patches/`, the one-line acceleration cap diff for `so_follower.py` (applied against LeRobot commit `3f2c29ef`, approximate upstream anchor `bf9d9a18`; full 40-character hashes are in the repository and in Table 7), the three-batch dataset merge recipe, the transfer notes between the ML Desktop and the Pi 5, and the earlier session logs that back the specific numbers in this paper. Command templates for recording, training, and rollouts are reproduced in the text and also live in the appendices of the companion papers [2, 3]. The 23 September custom remote-inference patch (`remote_inference_server.py`, `patch_sync_for_remote_inference.py`) is included as a documented fallback; the measurements in Section 4 use LeRobot's official asynchronous system instead.

References

[1] I. Radulov. *When Does a Trigger Reach a Robot Policy? Architecture, Fine-Tuning, and a Correctly Targeted Visual Backdoor.* Zenodo, 2026 (all versions). https://doi.org/10.5281/zenodo.23143960

[2] I. Radulov. *Distribution Mismatch, Lighting, and Background Generalization in a Low-Cost Imitation-Learning Pick-and-Place Policy.* Zenodo, 2026. https://doi.org/10.5281/zenodo.23212210

[3] I. Radulov. *Field Notes from a Low-Cost Robot Learning Lab: Hardware Churn, Software Whiplash, and What Actually Went Wrong.* Zenodo, 2026. https://doi.org/10.5281/zenodo.23259697

[4] T. Z. Zhao, V. Kumar, S. Levine, C. Finn. *Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware.* Robotics: Science and Systems, 2023. arXiv:2304.13705.

[5] M. Shukor et al. *SmolVLA: A Vision-Language-Action Model for Affordable and Efficient Robotics.* arXiv:2506.01844, 2025.

[6] K. Black, M. Y. Galliker, S. Levine. *Real-Time Execution of Action Chunking Flow Policies.* arXiv:2506.07339, 2025.

[7] K. Sendai, M. Alvarez, T. Matsushima, Y. Matsuo, Y. Iwasawa. *Leave No Observation Behind: Real-time Correction for VLA Action Chunks.* arXiv:2509.23224, 2025.

[8] Lian, Yu, Lin et al. *LangForce: Bayesian Decomposition of Vision Language Action Models via Latent Action Queries.* arXiv:2601.15197 (ICML 2026).

[9] A. Shaw, C. Liu, J. Costa, R. Gray, A. Skowronek, K. Diaz, N. Bui, N. Correll. *Cutting the Cord: System Architecture for Low-Cost, GPU-Accelerated Bimanual Mobile Manipulation.* arXiv:2603.09051, 2026.

zenodo:10.5281/zenodo.23918522 [cs.RO] Oct 2026 DOI: 10.5281/zenodo.23918522

[10] A. Kiviranta. *Deployment of a small vision-language-action model for low-cost manipulation: SmolVLA finetuned for SO-101.* Bachelor's thesis, Tampere University, October 2025. https://trepo.tuni.fi/bitstream/handle/10024/231225/KivirantaArttu.pdf

zenodo:10.5281/zenodo.23918522 [cs.RO] Oct 2026 DOI: 10.5281/zenodo.23918522